Python Interview Questions & Answers

The most frequently & commonly asked questions with detailed answers and code examples

24 essential questions • 7 topic areas • A complete prep guide

Contents

1. Python Fundamentals

What is Python and what are its key features?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum. Its key features are: it is **interpreted** (executed line by line, no separate compile step), **dynamically typed** (variable types are decided at runtime), **object-oriented**, has **automatic memory management** with garbage collection, an extensive **standard library**, and a readable, indentation-based syntax. It is portable and runs on almost every platform.

Is Python compiled or interpreted?

Both, in a sense. Python source code is first **compiled to bytecode** (the .pyc files), and that bytecode is then executed by the Python Virtual Machine (PVM). From the developer's point of view, it behaves like an interpreted language because there is no explicit compilation step.

What is PEP 8?

PEP 8 is the official **style guide** for writing Python code. It defines conventions for indentation (4 spaces), naming (snake_case for variables/functions, PascalCase for classes), maximum line length, and import ordering. Following PEP 8 makes code consistent and readable across teams. Tools like `flake8`, `black` and `pylint` help enforce it.

2. Data Types & Structures

What is the difference between a list and a tuple?

A **list** is mutable (can be changed after creation) and uses square brackets `[ ]`. A **tuple** is immutable (cannot be changed) and uses parentheses `( )`. Tuples are slightly faster, use less memory, and can be used as dictionary keys; lists cannot. Use a tuple for fixed collections of items and a list for collections that change.

```
>>> my_list = [1, 2, 3]
>>> my_list[0] = 99      # works fine
>>> my_tuple = (1, 2, 3)
>>> my_tuple[0] = 99     # TypeError: 'tuple' object does not
                         # support item assignment
```

What is the difference between '==' and 'is'?

== compares **values** (are the two objects equal?). `is` compares **identity** (do the two names point to the exact same object in memory?). Two different objects can be equal with == but still not `is` each other.

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a == b      # True  -> same values
>>> a is b      # False -> different objects in memory
```

What are mutable and immutable types in Python?

Mutable objects can be changed in place: `list`, `dict`, `set`. **Immutable** objects cannot be changed after creation: `int`, `float`, `str`, `tuple`, `frozenset`. When you 'change' an immutable object you actually create a new object. This distinction matters for function arguments and dictionary keys (keys must be immutable).

How is a dictionary different from a set?

A **dictionary** stores key-value pairs `{key: value}` and provides fast O(1) lookups by key. A **set** stores only unique, unordered values `{a, b, c}` and is used for membership tests and removing duplicates. Both are built on hash tables, so their elements/keys must be hashable (immutable).

3. Functions & Arguments

What are *args and **kwargs?

They let a function accept a variable number of arguments. `*args` collects extra **positional** arguments into a tuple, and `**kwargs` collects extra **keyword** arguments into a dictionary. They are useful when you don't know in advance how many arguments will be passed.

```
def demo(*args, **kwargs):
    print(args)     # tuple of positional args
    print(kwargs)   # dict of keyword args

demo(1, 2, name='Sam', age=30)
# (1, 2)
# {'name': 'Sam', 'age': 30}
```

What is a lambda function?

A lambda is a small, anonymous, single-expression function defined with the `lambda` keyword. It is handy for short throwaway functions, often passed to `map()`, `filter()`, or `sorted()`. It can take any number of arguments but only one expression, whose value is returned automatically.

```
>>> square = lambda x: x * x
>>> square(5)            # 25
>>> sorted(data, key=lambda p: p['age'])
```

What is the difference between pass-by-value and pass-by-reference in Python?

Python uses what is best called **pass-by-object-reference** (or 'pass-by-assignment'). The function receives a reference to the same object. If the object is **mutable** (e.g. a list) and you modify it in place, the change is visible outside. If the object is **immutable** (e.g. an int or string) or you rebind the name, the caller's variable is unaffected.

4. Comprehensions, Iterators & Generators

What is a list comprehension?

A list comprehension is a concise way to build a list from an iterable in a single readable line. It is usually faster and cleaner than an equivalent for-loop with `append()`. The same idea works for dictionaries and sets.

```
# Squares of even numbers 0-9
squares = [x*x for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]

# Dict comprehension
mapping = {x: x*x for x in range(5)}
```

What is the difference between an iterator and a generator?

An **iterator** is any object that implements `__iter__()` and `__next__()`. A **generator** is a simpler way to create an iterator using a function with the `yield` keyword. Generators are **lazy**: they produce values one at a time on demand instead of building the whole sequence in memory, which makes them very memory-efficient for large or infinite sequences.

```
def count_up(n):
    i = 0
    while i < n:
        yield i        # pauses here, resumes on next()
        i += 1

for num in count_up(3):
    print(num)         # 0, 1, 2
```

What does the yield keyword do?

`yield` turns a function into a generator. Each call to the generator runs until it hits a `yield`, returns that value, and **pauses** its state. The next request resumes right after the yield. Unlike `return`, it does not destroy the function's local state, so it can produce a stream of values over time.

5. Object-Oriented Programming

What is 'self' in Python classes?

`self` refers to the **current instance** of the class. It is the first parameter of instance methods and lets each method access that object's attributes and other methods. It is passed automatically by Python when you call a method on an instance, so you never pass it explicitly.

What is the purpose of __init__?

`__init__` is the **constructor** method. It runs automatically when a new object is created and is used to initialise the object's attributes. It is one of many 'dunder' (double-underscore) methods Python calls under the hood.

```
class Person:
    def __init__(self, name, age):
        self.name = name      # instance attribute
        self.age = age

p = Person('Alice', 28)
print(p.name)                 # Alice
```

Explain the four pillars of OOP in Python.

Encapsulation – bundling data and methods together and restricting direct access (via naming conventions like `_protected` and `__private`). **Inheritance** – a class can derive from another and reuse its behaviour. **Polymorphism** – the same method name behaves differently across classes (e.g. `len()` works on many types). **Abstraction** – hiding complex implementation behind a simple interface, often via abstract base classes.

What is the difference between class variables and instance variables?

Class variables are shared by all instances and defined directly in the class body. **Instance variables** are unique to each object and usually set inside `__init__` with `self`. Changing a class variable affects every instance that hasn't overridden it.

6. Advanced Concepts

What is a decorator?

A decorator is a function that takes another function and **extends or modifies its behaviour** without changing its source code. It is applied with the @ syntax above a function. Decorators are commonly used for logging, timing, access control, and caching.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print('Before call')
        result = func(*args, **kwargs)
        print('After call')
        return result
    return wrapper

@my_decorator
def greet():
    print('Hello!')
```

What is the Global Interpreter Lock (GIL)?

The GIL is a mutex in CPython that allows only **one thread to execute Python bytecode at a time**, even on multi-core CPUs. This simplifies memory management but means CPU-bound multithreading does not give true parallelism. For CPU-bound work use `multiprocessing` (separate processes); for I/O-bound work, threads and `asyncio` still help.

What is the difference between a shallow copy and a deep copy?

A **shallow copy** (`copy.copy()`) creates a new outer object but still references the same nested objects, so changes to nested items affect both copies. A **deep copy** (`copy.deepcopy()`) recursively copies everything, producing a fully independent object.

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)
shallow[0][0] = 99   # also changes original
deep[0][0] = 0       # original untouched
```

What is a context manager / what does the 'with' statement do?

A context manager handles **setup and cleanup** around a block of code, guaranteeing resources are released even if an error occurs. The `with` statement is the common way to use one, most often for files. Behind the scenes it calls `__enter__()` at the start and `__exit__()` at the end.

```
with open('data.txt') as f:
    content = f.read()
# file is closed automatically here,
# even if an exception was raised
```

7. Error Handling & Memory

How does exception handling work in Python?

Python uses `try / except / else / finally`. Code that might fail goes in `try`; `except` catches specific exceptions; `else` runs only if no exception occurred; and `finally` always runs (used for cleanup). Catch specific exceptions rather than a bare `except`.

```
try:
    result = 10 / divisor
except ZeroDivisionError:
    print('Cannot divide by zero')
else:
    print('Success:', result)
finally:
    print('Always runs')
```

How does memory management work in Python?

Python manages memory automatically with a **private heap**. Objects are tracked by **reference counting** – when an object's reference count drops to zero it is freed. A **cyclic garbage collector** additionally detects and cleans up reference cycles. Developers do not manually allocate or free memory.

What are Python namespaces and the LEGB rule?

A namespace maps names to objects. When you reference a name, Python searches scopes in the order **LEGB**: **L**ocal (inside the current function), **E**nclosing (any outer functions), **G**lobal (module level), and **B**uilt-in (e.g. `print`, `len`). The first match wins.